



Security Assessment

Nearly.Trade

Verified on October 2, 2026

SUMMARY

Project

Nearly.Trade

CHAIN

NEAR

METHODOLOGY

Manual & Automatic Analysis

FILES

Factory & Locker

DELIVERY

October 2, 2026

TYPE

Standard Audit



7

Total Findings

0

Critical

1

Major

2

Medium

3

Minor

1

Informational

0

Resolved

0 Critical

An exposure that can affect the contract functions in several events that can risk and disrupt the contract

1 Major

An opening & exposure to manipulate the contract in an unwanted manner

2 Medium

An opening that could affect the outcome in executing the contract in a specific situation

3 Minor

An opening but doesn't have an impact on the functionality of the contract

1 Informational

An opening that consists information but will not risk or affect the contract

0 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS

⚠️ PENDING FIXES

TABLE OF CONTENTS | Nearly.Trade

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details

Findings

Issues
Rust Attacks
Fixes & Recommendation
Audit Comments

DISCLAIMER | Nearly.Trade

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

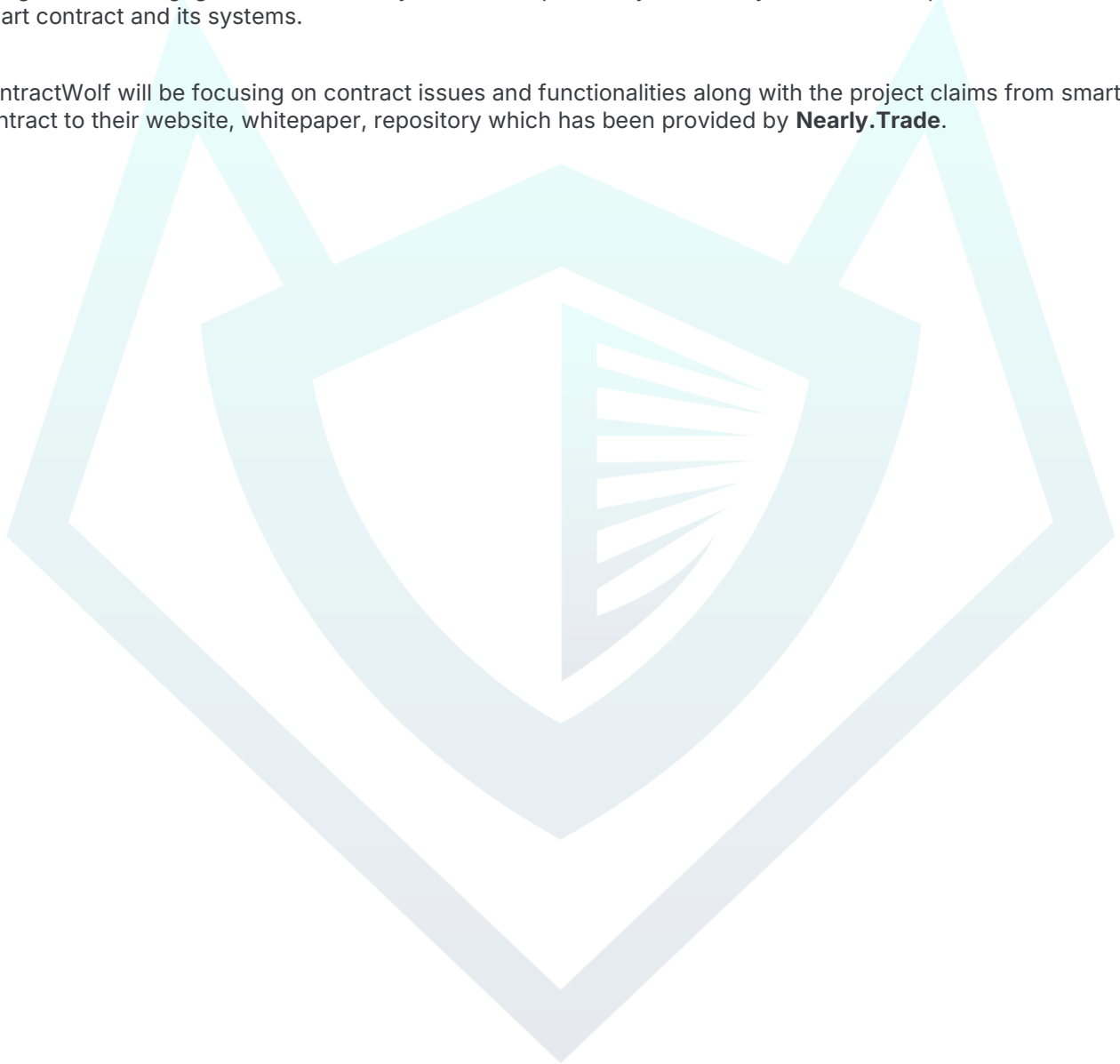
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | Nearly.Trade

Nearly.Trade team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Nearly.Trade**.



AUDITING APPROACH | Nearly.Trade

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | Nearly.Trade



nearly.trade is a token launchpad on NEAR. Every launch creates a token and opens a single sided liquidity position on Rhea's DCL, held by our locker contract. Creators earn 80% of pool fees. Fees on \$NEARLY are used to buy back and burn the token.

SOURCE

Source

Local Files (contracts/factory, contracts/locker3)

FINDINGS

Nearly.Trade



7

0

1

2

3

1

0

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for Nearly.Trade through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
ADMIN-KEY-01	Factory Owner Is a Full-Access Key With Instant Upgrade	factory/lib.rs	Major	● Pending
RCW-015	Predictable Token Accounts Let Anyone Fail a Launch	factory/lib.rs	Medium	● Pending
TAX-SELLER-01	Tax Proceeds Are Whatever the Off-Chain Seller Reports	factory/lib.rs	Medium	● Pending
RCW-014	Holder Payouts Trust an Unverifiable Off-Chain List	factory/lib.rs	Minor	● Pending
LOCKER-SUPPLY-01	Locker Withdrawal Can Take a Launch's Unplaced Supply	locker3/lib.rs	Minor	● Pending

WNEAR-
SWEEP-01

Unwrap Failures Leave
Creator wNEAR Open
to the Protocol Sweep

factory/lib.rs

Minor

● Pending

BUYBACK-
EXCESS-01

Buyback Fill Above
104% of min_out Is
Never Burned

factory/lib.rs

Informational

● Pending



RUST ATTACKS | Nearly.Trade

Rust Smart Contract Weakness Classification and Test Cases

ID	Name	Description	Status
RCW-001	Reentrancy	A malicious contract calls back into the calling contract before the first invocation of the function is finished.	● Passed
RCW-002	Undefined behavior	The Rust reference contains a non-exhaustive list of behaviors considered undefined in Rust.	● Passed
RCW-003	Integer overflow & underflow	Overflow or underflow of mathematical operations inside the Rust smart contract.	● Passed
RCW-004	Out of bounds read/write	The contract or function reads data past the end, or before the beginning, of the intended buffer.	● Passed
RCW-005	Memory corruption	Wrong usage of the memory model throughout the contract or within its functions.	● Passed
RCW-006	Typographical error	Unintended error in contract and function names, code, and arithmetic inputs.	● Passed
RCW-007	Hash collisions with multiple arguments	If used incorrectly, triggers a hash collision while calling a function within a function.	● Passed
RCW-008	Account confusion	Failure to verify account ownership or types, leading to unauthorized access.	● Passed
RCW-009	Insufficient rent exemption	Accounts not properly funded for rent exemption, risking purge.	● Passed
RCW-010	Unverified PDA derivations	Not validating Program Derived Address (PDA) seeds or bump.	● Passed
RCW-011	Missing authority checks	Unauthorized token minting via unverified authorities, and improper freeze authorities.	● Passed
RCW-012	Arithmetic precision loss	Truncation errors in financial calculations and default round-offs (e.g. 0.999 → 0).	● Passed
RCW-013	Compute unit exhaustion	Unbounded loops causing transaction failures.	● Passed
RCW-014	Reward distribution flaws	Inaccurate reward pro-rata calculations.	● Not Passed
RCW-015	Slot-based race conditions	Front-running via transaction ordering.	● Not Passed
RCW-016	Uncontrolled allocations	Memory allocation without size limits.	● Passed

FIXES & RECOMMENDATION

ADMIN-KEY-01 | Factory Owner Is a Full-Access Key With Instant Upgrade

contracts/factory/src/lib.rs

Every owner method checks `predecessor == owner_id`, and `owner_id` is the factory account itself, which on October 2, 2026 still held a full-access key beside seven function-call keys. Through that key, upgrade switches the factory to any global contract at once, with no timelock or multisig. The factory holds every creator, holder and router balance and is the tax admin of every token, so one compromised key reaches all of it.

```
// factory/lib.rs - L2673 any code, live in the same receipt
pub fn upgrade(&mut self, code_hash: Base58CryptoHash, migrate: bool) -> Promise {
    self.assert_owner();
    let hash: CryptoHash = code_hash.into();
    let p = Promise::new(env::current_account_id()).use_global_contract(hash);
    ...
}

// factory/lib.rs - L3183 the owner is this account, driven by its keys
fn assert_owner(&self) {
    require!(env::predecessor_account_id() == self.owner_id, "owner only");
}
```

Recommendation

Delete the full-access key as planned, move owner_id to a multisig or DAO account, and put upgrade behind a proposal that executes only after a public delay, so creators and holders can withdraw before new code takes over their balances.

```
const UPGRADE_DELAY_MS: u64 = 48 * 60 * 60 * 1000;
const UPGRADE_KEY: &[u8] = b"upg";

pub fn propose_upgrade(&mut self, code_hash: Base58CryptoHash) {
    self.assert_owner();
    let hash: CryptoHash = code_hash.into();
    let mut v = hash.to_vec();
    v.extend_from_slice(&env::block_timestamp_ms().to_le_bytes());
    env::storage_write(UPGRADE_KEY, &v);
    emit("upgrade_proposed", &format!(r#"{"code_hash":"{}}"}"#, hex32(hash)));
}

pub fn upgrade(&mut self, code_hash: Base58CryptoHash, migrate: bool) -> Promise {
    self.assert_owner();
    let p = env::storage_read(UPGRADE_KEY).expect("no proposal");
    let hash: CryptoHash = code_hash.into();
    let at = u64::from_le_bytes(p[32..40].try_into().unwrap());
    require!(p[..32] == hash[..], "not the proposed code");
    require!(env::block_timestamp_ms() >= at + UPGRADE_DELAY_MS, "timelocked");
    env::storage_remove(UPGRADE_KEY);
    // ... use_global_contract(hash) as today
}
```

RCW-015 | Predictable Token Accounts Let Anyone Fail a Launch

contracts/factory/src/lib.rs

Token accounts are named from a public per-symbol counter (pepe, pepe-2, ...), so the next account for any symbol is known in advance. Anyone can call Rhea's `create_pool` for it first, at another start point, for about 0.1 NEAR. The launch then fails in `on_pool_checked`: the creator loses the token storage already spent and the refused pool deposit, which comes back to the factory instead. Repeated, this blocks popular symbols outright.

```
// factory/lib.rs - L792 the next account for a symbol is public state
let mut taken = slug_count(&slug);
if taken == 0 && RESERVED_SLUGS.contains(&slug.as_str()) { taken = 1; }
set_slug_count(&slug, taken + 1);
let token: AccountId = format!(
    "{}.{}", short_name(&slug, taken), env::current_account_id()
).parse().expect("token account id");

// factory/lib.rs - L1066 a pool at another point fails the launch
Ok(Some(_)) => self.internal_mark_failed(id, 1, "pool exists at another price"),
```

Recommendation

Make the account unknowable until the launch receipt runs, for example with a short suffix from `env::random_seed()`, so pools cannot be prepared in advance, and refund the refused `create_pool` deposit to the creator whenever a launch fails or adopts an existing pool.

```
// factory/lib.rs - launch(): a suffix nobody knows before this receipt runs
let seed = env::random_seed();
let tag: String = seed[..2].iter().map(|b| format!("{:02x}", b)).collect();
let name = format!("{}-{}", short_name(&slug, taken), tag);
let token: AccountId = format!("{}.{}", name, env::current_account_id())
    .parse().expect("token account id");

// factory/lib.rs - on_pool_checked: the deposit came back here, it is the creator's
Ok(Some(_)) => {
    let fee = DCL_POOL_CREATE.as_yoctonear();
    self.internal_refund_creator(id, &l.creator, fee).detach();
    self.internal_mark_failed(id, 1, "pool exists at another price")
}
```

TAX-SELLER-01 | Tax Proceeds Are Whatever the Off-Chain Seller Reports

contracts/factory/src/lib.rs

process_tax hands every tax token that is not burned (platform, creator and holder shares alike) to tax_seller, feeswap.near, whose key lives on a server.

internal_tax_settle then books whatever NEAR or pair token the seller attaches as the proceeds, checking only sold <= selling, and min_out is accepted and ignored. A compromised or careless seller can sell at any price or keep the proceeds, and nothing on chain shows it.

```
// factory/lib.rs - L1774 the keeper's floor is discarded
let _ = (min_out, to_seller);
...
let seller = tax_seller().expect("no tax seller set");
add_counter(&tax_selling_key(launch_id.0), sell);

// factory/lib.rs - L3262 any proceeds are accepted for a bounded `sold`
let held = read_counter(&tax_selling_key(launch_id.0));
require!(sold > 0 && sold <= held, "sold is more than the seller holds for this launch");
require!(got > 0, "no proceeds");
```

Recommendation

Sell on chain with the balance-difference swap buyback already uses, or keep the seller but make min_out binding: store it as the floor of the slice and refuse a settlement whose proceeds fall below that floor pro rata to sold.

```
fn tax_floor_key(id: u64) -> Vec<u8> { [b"tf:".as_slice(), &id.to_le_bytes()].concat() }

// process_tax: the keeper's fresh quote becomes the slice's floor
require!(min_out.0 > 0, "min_out required");
add_counter(&tax_floor_key(launch_id.0), min_out.0);

// internal_tax_settle: proceeds must clear the floor for what was sold
let floor = mul_div(read_counter(&tax_floor_key(launch_id.0)), sold, held);
require!(got >= floor, "proceeds below the keeper's floor");
sub_counter(&tax_floor_key(launch_id.0), floor);
```

RCW-014 | Holder Payouts Trust an Unverifiable Off-Chain List

contracts/factory/src/lib.rs

pay_holders and pay_tax_holders pay whatever list the keeper passes: any unique accounts, any amounts, bounded only by the launch's bucket. The pro-rata split is computed off chain and nothing ties a payout to a holder's balance or a snapshot. A faulty or compromised keeper key (on mainnet a function-call key holds pay_tax_holders) can send a holders bucket to arbitrary accounts.

```
// factory/lib.rs - L3150 only count, uniqueness and the total are checked
require!(!payouts.is_empty() && payouts.len() <= max, "too many payouts per call ...");
require_unique_payees(&payouts);
let total = payouts.iter()
    .fold(0u128, |a, (_, v)| a.checked_add(v.0).expect("overflow"));
...
let have = read_counter(&bucket);
require!(
    total > 0 && total <= have,
    "payouts exceed what this launch earned for its holders"
);
```

Recommendation

Commit each payout round on chain before paying it: the keeper posts a Merkle root of the holder snapshot and its total, and each holder claims their own share with a proof, so a payout can only reach an account the published snapshot entitles.

```
// keeper: publish the round (root of sha256("account:amount") leaves) and its total
pub fn set_holder_round(&mut self, launch_id: U64, root: Base58CryptoHash, total: U128) {
    self.assert_keeper();
    require!(total.0 > 0 && total.0 <= read_counter(&tax_holders_key(launch_id.0)));
    // store root + total, open the round
}

// holder: claim with a proof; the leaf binds the caller and the amount
pub fn claim_holder_share(
    &mut self, launch_id: U64, amount: U128, proof: Vec<Base58CryptoHash>,
) {
    let me = env::predecessor_account_id();
    let leaf = env::sha256_array(format!("{:?}", me, amount.0).as_bytes());
    require!(verify_proof(&proof, leaf, round_root(launch_id.0)), "not in this round");
    require!(!claimed(launch_id.0, &me), "already claimed");
    // mark claimed, debit the round, pay `me`
}
```

LOCKER-SUPPLY-01 | Locker Withdrawal Can Take a Launch's Unplaced Supply

contracts/locker3/src/lib.rs, contracts/factory/src/lib.rs

on_admin_asset lets the factory withdraw any of the locker's DCL balance of a token above dcl_reserved, which counts only released fees. A launch whose supply reached the exchange without a position (the case on_asset places with add_liquidity) has its whole supply there, unreserved, so the owner can pull it into protocol_token_fees. The position can then never open, and cancel_stuck_launch keeps the launch stuck until the owner steps in.

```
// locker3/lib.rs - L982 only released fees are protected
let reserved = self.dcl_reserved.get(&token).copied().unwrap_or(0);
require!(
    amount.0 <= admin_free(bal.0, reserved),
    "that is more than the DCL balance not owned by launch fees"
);

// locker3/lib.rs - L1470 yet a launch's whole supply can wait in that balance
fn asset_step(dcl_balance: u128, supply: u128, zapped_here: bool) -> AssetStep {
    if dcl_balance >= supply {
        AssetStep::AddOnly
    }
    ...
}
```

Recommendation

In the next locker build (lock_5 cannot change), reserve a launch's supply from the moment add or add_buy sends it until record books the position, releasing it only when the factory reports the launch failed, so withdraw_dcl_asset reaches true remainders only.

```
// add / add_buy: the supply is this launch's until its position is recorded
self.supply_reserved.insert(args.token.clone(), supply);

// record(): the position holds it now
self.supply_reserved.remove(&args.token);

// on_admin_asset: neither fees nor a waiting supply count as free
let reserved = self.dcl_reserved.get(&token).copied().unwrap_or(0)
    + self.supply_reserved.get(&token).copied().unwrap_or(0);
```

WNEAR-SWEEP-01 | Unwrap Failures Leave Creator wNEAR Open to the Protocol Sweep

contracts/factory/src/lib.rs

Three paths leave wNEAR on the factory with no record of whose it is: an exact dev buy's excess whose unwrap fails (on_dev_refund_unwrapped), a buyback's unused input whose unwrap fails (on_bb_unwrapped), and a refused buy's uw: record that release_dev_buy clears. sweep_wnear is held back only by uw: records, so the next sweep books the creator's or the burn bucket's wNEAR to protocol_fees.

```
// factory/lib.rs - L1276 the creator's excess stays as untracked wNEAR
if r.is_err() {
    emit("dev_buy_refund_unwrap_failed", &format!("{}", id, creator, amount));
    return false;
}

// factory/lib.rs - L2281 only `uw:` records hold a sweep back
require!(
    read_counter(UW_TOTAL_KEY) == 0,
    "a refused dev buy's wNEAR is still to be unwrapped: resume it first"
);
```

Recommendation

Record every unwrap failure as wNEAR owed to its launch (counted in uwt under a per-launch key, like uw:), stop release_dev_buy from dropping it, and give each record a retry that pays the creator or re-credits the mode bucket, so a sweep only takes protocol wNEAR.

```
// on_dev_refund_unwrapped / on_bb_unwrapped: keep the wNEAR booked to its launch
if r.is_err() {
    add_counter(UW_TOTAL_KEY, amount.0);
    add_counter(&wnear_owed_key(id), amount.0); // paid out by a retry method
    return false;
}

// release_dev_buy: move the refused buy's record instead of dropping it
let uw = read_u128(&unwrap_pending_key(id)).unwrap_or(0);
if uw > 0 {
    env::storage_remove(&unwrap_pending_key(id));
    add_counter(&wnear_owed_key(id), uw); // still counted in uwt
}
```

BUYBACK-EXCESS-01 | Buyback Fill Above 104% of min_out Is Never Burned

contracts/factory/src/lib.rs

on_bb_after burns at most 104% of the keeper's min_out and leaves the rest of the measured balance difference where it is. If a fill beats the floor by more than 4% (a stale quote, a price move in the launch's favour), the extra tokens, bought with the launch's buyback NEAR, stay on the factory outside every bucket: never burned, never counted, and out of reach of any method.

```
// factory/lib.rs - L1700 the part above 104% of min_out is dropped
let diff = if ours {
    bal.map(|a| a.0.saturating_sub(before.0)).unwrap_or(0)
} else {
    0
};
let bought = match min_out {
    Some(m) => diff.min(mul_div(m.0, 104, 100)),
    None => diff,
};
```

Recommendation

Book the part of the difference above the cap to the launch's creator_token_fees, which a burn-mode launch can only burn (burn_token_fees), so every token a buyback buys is burned at once or stays burnable.

```
let cap = min_out.map_or(diff, |m| mul_div(m.0, 104, 100));
let bought = diff.min(cap);
if diff > bought {
    // a burn-mode launch can only burn this bucket (burn_token_fees)
    let mut l = self.rec(launch_id.0);
    l.creator_token_fees = U128(l.creator_token_fees.0 + (diff - bought));
    self.launches.insert(launch_id.0, l);
}
```

AUDIT COMMENTS | Nearly.Trade

Smart Contract audit comment for a non-technical perspective

- The factory account still holds a full-access key, and its owner can switch the contract to new code at once with no timelock - check ADMIN-KEY-01
- Tax tokens are sold by an off-chain seller whose reported proceeds the contract cannot verify - check TAX-SELLER-01
- Anyone can make a launch fail by creating its pool first at another price, and the creator loses the fees already spent - check RCW-015
- Holder rewards follow a list computed off chain that the contract cannot check against real balances - check RCW-014
- Every launch's liquidity is owned by a locker with no access keys and no method that removes liquidity
- Creators' fee balances can only be paid to the creator or their chosen fee wallet, and a pause never blocks a creator's own claim
- Dev buys carry a fixed minimum output and are always refunded to the creator, never to the caller
- The creator's share of pool fees is fixed at 80% for every launch and cannot change later
- Ownership moves only in two steps, and no method lets the owner mint tokens or move a launch's position



CONTRACTWOLF

Blockchain Security - Smart Contract Audits